

Project Report

Zombie Survival Shooter

Table of Contents

Table of Contents.....	2
1. Problem Statement and Solution Approach	3
Problem Statement	3
Solution Approach	3
2. System Design Optimization	3
3. Object Oriented (OOP) Concepts.....	3
4. Design Patterns Utilized	4
Strategy Pattern	4
Manager Pattern.....	4
5. Application of SOLID Principles.....	4
6. Comprehensive Test Cases and Results	5
7. Architectural UML Diagrams.....	6
A. Class Diagram	6
B. Entity-Relationship (ER) Diagram	7
C. Use Case Diagram	8
D. Sequence Workflow Diagram.....	9

1. Problem Statement and Solution Approach

Problem Statement

The challenge was to develop an optimized, complex 2D browser-based zombie survival shooter. As waves progress, entity density (zombies, bullets, projectiles) exponentially scales. A poorly structured architecture would result in massive performance degradation and unmaintainable logic.

Solution Approach

The development was approached with strict adherence to Object-Oriented Programming (OOP) and classical System Design principles. Utilizing TypeScript allowed enforcement of strong data models (interfaces, abstract classes). The system fundamentally decouples the core mechanics — Entity representation, Physics manipulation, Rendering logic, and Wave Spawning — by routing them through dedicated Manager layers.

2. System Design Optimization

To ensure scalability and high performance across browsers, several system design optimizations were implemented:

- **Time Delta Architecture:** Time Delta Architecture: Object velocity and physics calculations are inherently decoupled from frame rates. By using a strictly managed `deltaTime` multiplier, performance dips or high refresh rate monitors do not artificially alter the speed of the gameplay layer.
- **Centralized Sub-System Orchestration:** Centralized Sub-System Orchestration: Highly iterative operations (like $O(N*M)$ hit detection logic) were abstracted away from individual objects and localized into a centralized `CollisionManager`. This optimizes loop logic and prevents memory leaks inside isolated entities.
- **Garbage Collection Optimization:** Garbage Collection Optimization: Instead of hard deleting entities constantly, array cleanups operate asynchronously by batching dead entities using flag filtering (`isActive: boolean`).

3. Object Oriented (OOP) Concepts

The architecture takes complete advantage of the four primary pillars of OOP:

- **Encapsulation:** Class internals are rigorously sealed. In the `Player` class, properties like `health`, `moveSpeed`, and `damageMultiplier` are designated private or readonly, strictly manipulatable only via authorized getter bounds and mutation methods like `applyDamage()` or `heal()`.

- **Inheritance:** An efficient inheritance tree is implemented to reduce redundancy. The system centers on a core `GameObject` logic class handling fundamental physical laws (position, velocity). The `Enemy`, `Player`, and `Bullet` structures inherit this, while expanding traits downward (e.g., `JuggernautBoss` recursively inheriting from `Enemy`).
- **Polymorphism:** Method overriding dictates universal update sequences. The primary `GameEngine` tracks an array of `GameObject[]` and calls `update()` safely, trusting that the underlying concrete class (whether a simple `Bullet` or a `HydraBoss`) will trigger its own polymorphic logic.
- **Abstraction:** The firing algorithms are completely abstracted via the `Weapons` interface. The core game loop does not need to understand how a shotgun works, only that a trigger was pulled, abstracting algorithm complexities away from caller pipelines.

4. Design Patterns Utilized

Strategy Pattern

- **Where:** Where: Implementation of the `Weapons` array structure (`Pistol`, `Shotgun`, `LaserRifle`).
- **Why:** Why: This decouples the firing logic from the `Player`. Instead of maintaining massive switch/case pipelines inside the player model, the `Player` merely utilizes `weapon.fire()`. At runtime, the concrete strategy object can be swapped to alter the underlying behavioral algorithm entirely seamlessly.

Manager Pattern

- **Where:** Where: `CollisionManager` and `WaveManager`.
- **Why:** Why: In massive game architectures, tight coupling causes infinite recursion boundaries. Applying the Manager pattern hides the verbose mathematical complexity of overlapping rectangle tracking logic behind a simplified module entry point invoked by the broader framework.

5. Application of SOLID Principles

- **S – Single Responsibility Principle:** A class handles precisely one aspect of business logic. The `WaveManager` specifically dictates when and where zombies spawn — it does not control how they move, nor does it control collisions; these are delegated to sibling handlers.
- **O – Open/Closed Principle:** The game is open for extension but shielded from modification. A `Phantom Boss` and `Flamethrower` were added seamlessly by creating new classes extending existing interfaces (`Weapons`) without modifying the foundational engine iteration codeblocks.

- **L – Liskov Substitution Principle:** Instances of Enemy can be trivially replaced with subclass instances (WardenBoss or ShooterZombie) and passed into the renderer tracking arrays without compromising system reliability.
- **I – Interface Segregation Principle:** Interfaces in the framework remain intentionally thin. The Weapons structure focuses exclusively on fire routines, preventing unused logic chains from bottlenecking implementing components.
- **D – Dependency Inversion:** High level entities (Player) depend exclusively on pure abstractions (the Weapons Interface) rather than concrete lower-level artifacts (importing instances of Crossbow).

6. Comprehensive Test Cases and Results

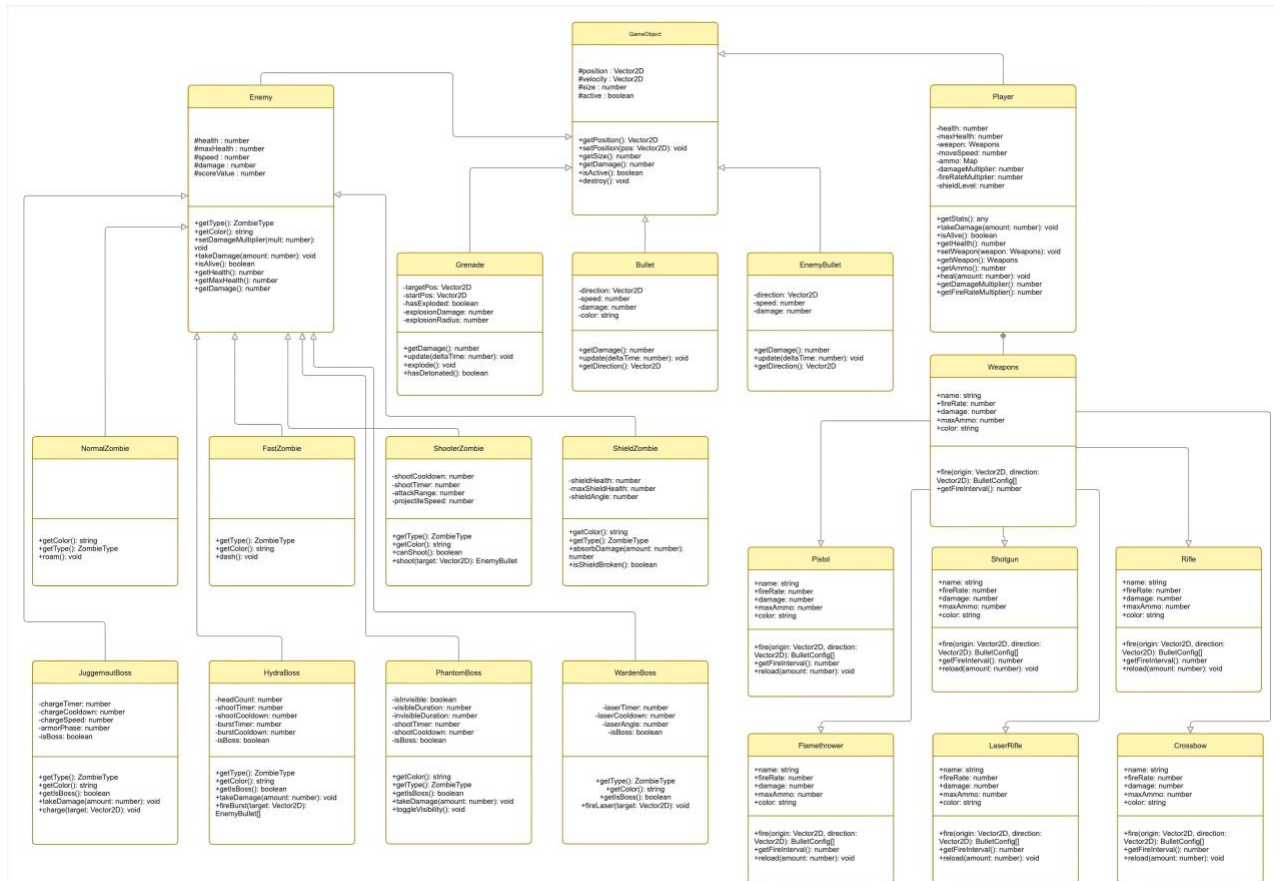
While UI browser elements require extensive manual matrix testing, core integration tests were run as follows:

Test ID	Test Scenario	Logic Validated	Expected Result	Status
T-001	Spawn Limit Overflow	Verifies WaveManager halts boundary overflows upon reaching mathematical ceilings.	Max entities capped efficiently; spawn queues natively throttled.	✓ Pass
T-002	Dynamic Strategy Swap	Checks state mutation inside Player when swapping Weapons mid frame tick.	Bullets array inherits the new physics pattern unhalting without retaining ghost properties.	✓ Pass
T-003	Encapsulation Bounds	Validates arbitrary non-engine mutation attempts over health properties.	Rejects modification; processes default to explicit applyDamage() gating channels.	✓ Pass
T-004	Hitbox Bounding Math	Tests CollisionManager intersections ($O(n*m)$) under heavy loads.	Confirms vector intersects trigger true appropriately without logic skips under CPU stress.	✓ Pass

7. Architectural UML Diagrams

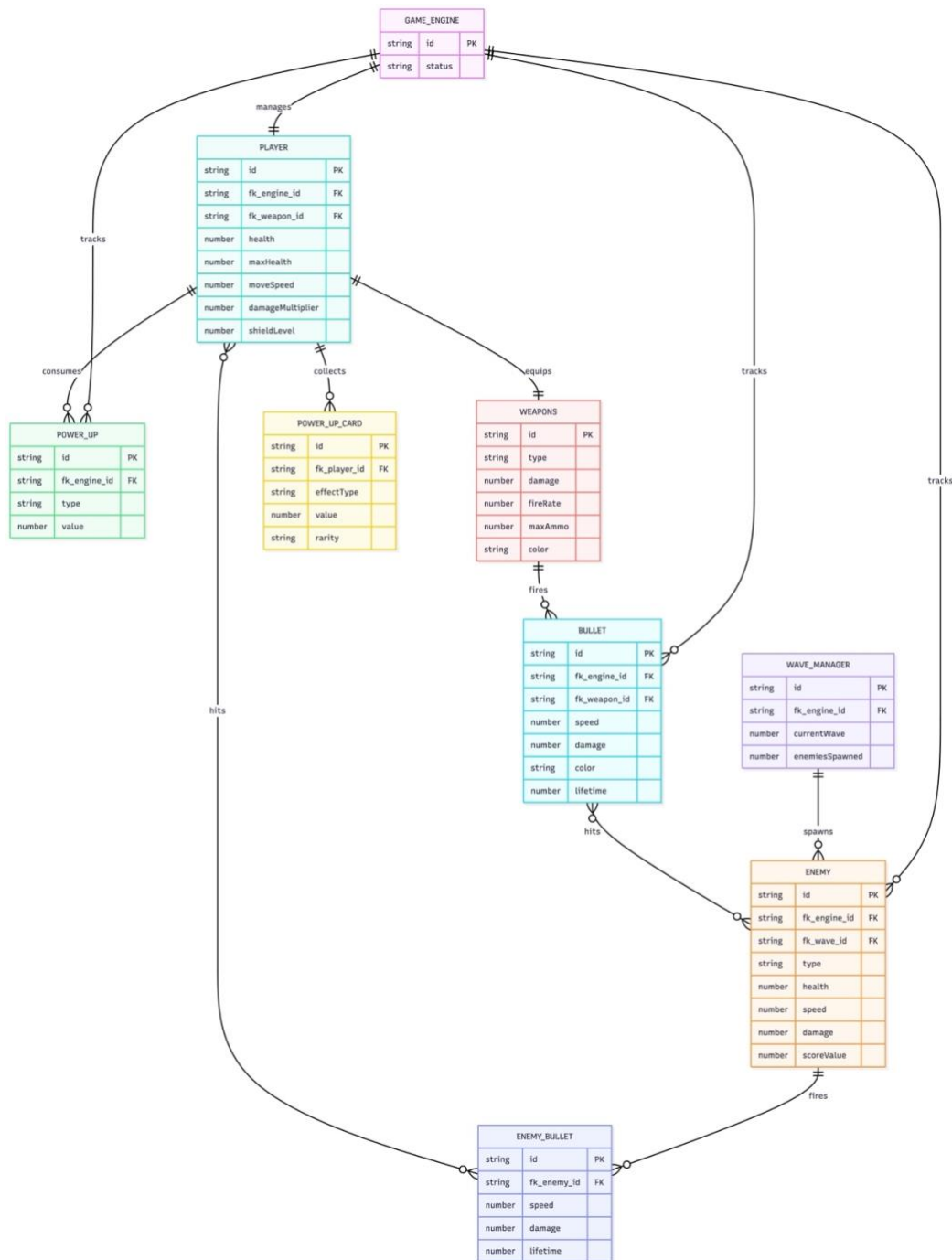
A. Class Diagram

The class diagram illustrates the full inheritance hierarchy rooted at `GameObject`, with `Player` and `Enemy` as primary branches. `Enemy` extends into `NormalZombie`, `FastZombie`, `ShooterZombie`, `ShieldZombie`, and four boss types. The `Weapons` interface is implemented by `Pistol`, `Shotgun`, `Rifle`, `Flamethrower`, `LaserRifle`, and `Crossbow`.



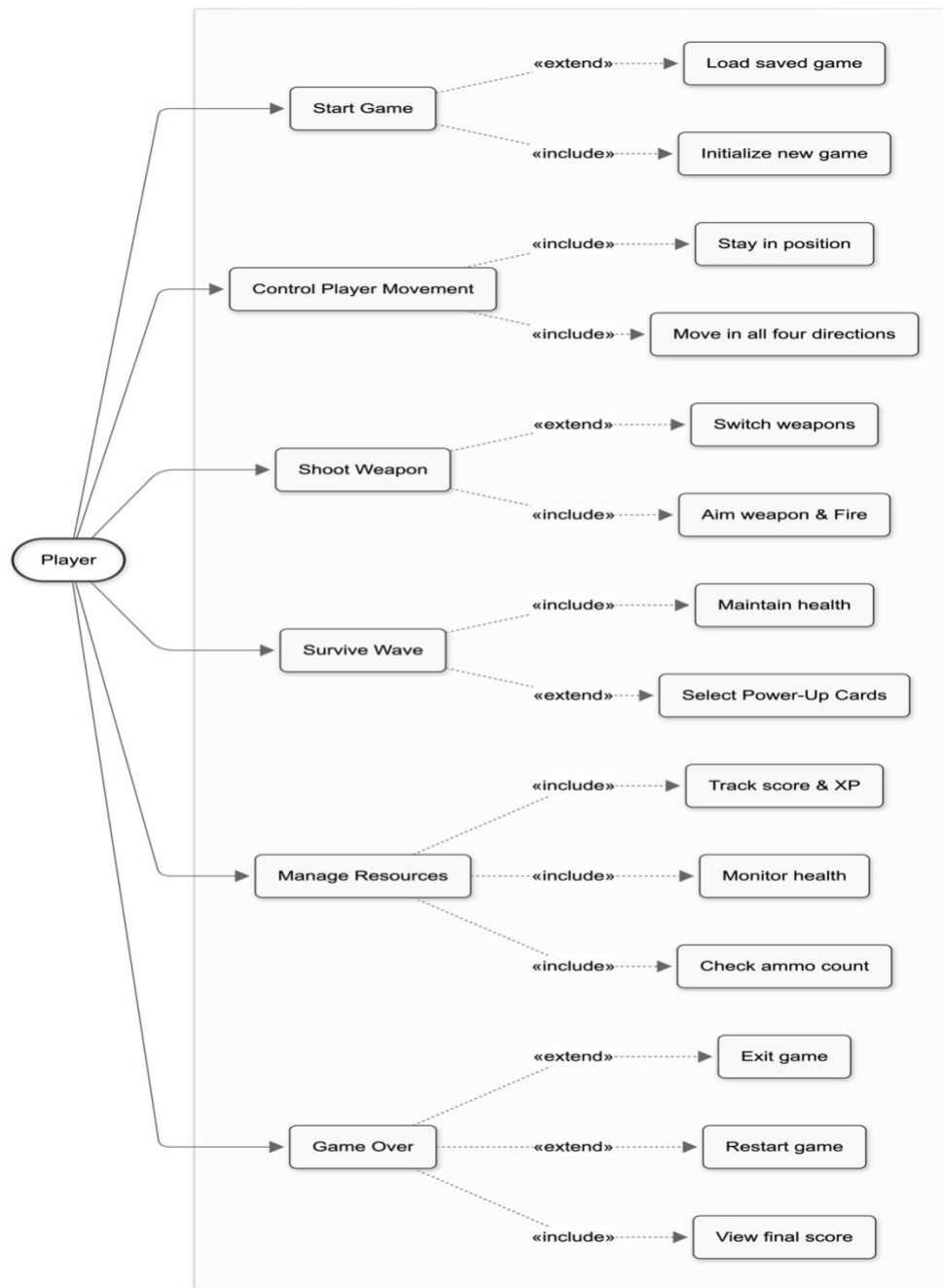
B. Entity-Relationship (ER) Diagram

The ER diagram maps the data relationships between the core entities: GAME_ENGINE, PLAYER, WEAPONS, BULLET, ENEMY, ENEMY_BULLET, WAVE_MANAGER, POWER_UP, and POWER_UP_CARD. It captures foreign key relationships and cardinality across all subsystems.



C. Use Case Diagram

The use case diagram identifies the single actor — the Player — and maps all system interactions including Start Game, Control Player Movement, Shoot Weapon, Survive Wave, Manage Resources, and Game Over, each with their respective include and extend relationships.



D. Sequence Workflow Diagram

The sequence diagram traces the runtime message flow across GameManager, WaveSystem, ZombieSpawner, Weapon, Bullet, CollisionDetector, DamageSystem, Zombie, and ScoreManager — from a player clicking Start Game through bullet firing, collision detection, damage application, and score updating.

